



Final Project – Event Management Platform

Task Overview

The objective of this project is to design and develop a full-stack Event Management Platform called Convene. The platform will allow users to create events, browse them, register as attendees, and view a dashboard of platform statistics. It will leverage **Node.js** and **Express.js** for the REST API, **MongoDB** with **Mongoose** for data persistence, and **Vanilla JavaScript** for the frontend Single-Page Application. The project is split into two repositories that communicate over HTTP using `fetch()` and JSON.

Detailed Requirements

A) Dashboard View

The main landing view of the application. It displays a live overview of platform statistics fetched from the backend API.

Features:

1. Platform Statistics:

- Total number of events on the platform
- Total upcoming events (date in the future)
- Total registrations across all events
- Most popular event (highest registration count)

2. Loading & Error States:

- Show a loading indicator while data is being fetched
- Show an error toast notification if the API call fails

JavaScript Requirements:

- Use `fetch()` to call `GET /api/dashboard` on page load.
- Dynamically render all stat values into the DOM.
- Handle loading and error states in the UI.

Backend Requirements:

- GET /api/dashboard — return { totalEvents, upcomingEvents, totalRegistrations, mostPopularEvent }.
- Use MongoDB aggregation to compute all stats in a single query.

CSS Requirements:

- Stat values displayed as styled cards in a responsive grid.
 - Responsive layout that adapts to mobile, tablet, and desktop.
-

B) Events List View

This view displays all events on the platform as browsable cards, with search and filter controls.

Features:

1. **Event Cards:**
 - Title, category, location, date, capacity
 - Number of registrations for each event
 - Edit and Delete action buttons on each card
2. **Search & Filter Controls:**
 - Search by event title.
 - Filter by category, location, and date
 - Clear button to reset all active filters

JavaScript Requirements:

- Use fetch() to call GET /api/events and render results as cards.
- Pass query params (?search=, ?category=, ?location=, ?date=) to the API.
- Clicking a card navigates to the Event Detail view without a page reload.
- Clicking Delete shows a confirmation modal before sending DELETE /api/events/:id.

Backend Requirements:

- GET /api/events — return all events; support ?search, ?category, ?location, ?date query params.
- DELETE /api/events/:id — delete the event and all its associated registrations.

CSS Requirements:

- Events displayed in a responsive card grid.
 - Responsive design for mobile and desktop.
-

C) Event Detail View

This view shows a single event's complete information and its list of registered attendees, along with a registration form.

Features:

1. Event Information:

- Full event details: title, category, location, date & time, capacity, description
- Remaining spots (capacity minus current registrations count)

1. Attendees List:

- Display names and emails of all registered attendees

2. Registration Form:

- Input fields: Name and Email
- Submit button to register for the event
- Form is disabled and shows a message if the event is at full capacity

JavaScript Requirements:

- Use fetch() to call GET /api/events/:id and GET /api/events/:id/attendees.
- Render remaining spots dynamically from capacity and registrationsCount.
- Submit registration form via POST /api/events/:id/register.
- Refresh the attendees list and remaining spots after a successful registration.
- Allow cancelling a registration via DELETE /api/events/:id/registrations/:id.

Backend Requirements:

- GET /api/events/:id — return event with registrationsCount.
- GET /api/events/:id/attendees — return list of registered attendees.
- POST /api/events/:id/register — register attendee; prevent duplicate emails.
- DELETE /api/events/:id/registrations/:registrationId — cancel a registration.

CSS Requirements:

- Event details and attendees list displayed in a clear, readable layout.
 - Registration form styled with proper spacing and validation error states.
-

D) Create / Edit Event View

A shared form view used both to create a new event and to edit an existing one. In edit mode the form is pre-filled with the existing event data.

Features:

1. Form Fields:

- Title
- Category
- Location
- Date & Time
- Capacity
- Description

2. Validation & Feedback:

- Client-side validation before submitting (all fields required, capacity > 0)
- Field-level error messages shown inline
- On success: navigate to the Events List and show a success toast

3. Edit Mode:

- Pre-fill all fields with the existing event's data
- Submit sends PUT /api/events/:id instead of POST /api/events

JavaScript Requirements:

- Use `fetch()` to submit `POST /api/events` (create) or `PUT /api/events/:id` (edit).
- Detect edit mode by reading the event ID from the app state or URL hash.
- Show field-level error messages returned from the server's validation response.
- Navigate to the Events List view and display a success toast on completion.

Backend Requirements:

- `POST /api/events` — create a new event; validate all fields server-side.
- `PUT /api/events/:id` — update an event; return the updated document.
- Return structured validation errors: `{ field, message }` array on failure.

CSS Requirements:

- Form styled with clear spacing, labels, and input focus states.
- Inline field-level error messages styled in red beneath each invalid input.
- Responsive mobile-friendly layout.

E) Backend API

The backend is a RESTful API built with Node.js and Express, connected to a MongoDB database via Mongoose. All responses use a consistent JSON envelope.

Features:**1. Events Endpoints:**

- `GET /api/events` — list all events with optional query filters
- `POST /api/events` — create a new event
- `GET /api/events/:id` — get a single event with `registrationsCount`
- `PUT /api/events/:id` — update an event
- `DELETE /api/events/:id` — delete event and all its registrations

2. Registration Endpoints:

- `POST /api/events/:id/register` — register an attendee (name + email)
- `GET /api/events/:id/attendees` — list all attendees for an event
- `DELETE /api/events/:id/registrations/:registrationId` — cancel a registration

3. Dashboard & Health Endpoints:

- GET /api/dashboard — return platform-wide stats
- GET /api/health — health check returning { success: true }

Node.js / Express Requirements:

- All responses follow the envelope: { success: true, data: ... } or { success: false, message: "..." }.
- Handle 404 (not found) and 500 (server error) with global error-handler middleware.
- Enable CORS so the frontend (<http://localhost:5500>) can communicate with the API.

MongoDB / Mongoose Requirements:

- Define an Event schema: title, category, location, date, capacity, description.
- Define a Registration schema: eventId (ref), name, email, registeredAt.
- Prevent duplicate registration by enforcing a unique index on { eventId, email }.
- Use MongoDB aggregation in the dashboard controller to compute all stats efficiently.

General Backend Requirements:

- Backend runs on port 5000 by default (configurable via PORT env var).
- All sensitive config (MONGO_URI, PORT, CLIENT_ORIGIN) stored in .env; never hardcoded.
- .env listed in .gitignore; .env.example committed with placeholder values.
- npm start runs the production server; npm run dev.

F) Navigation & SPA Routing

All views are part of a Single-Page Application. A top navigation bar allows switching between views without full page reloads.

Features:

1. Navigation Bar:

- Links to all four views: Dashboard, Events, New Event
- Active link highlighted to indicate the current view

2. SPA View Switching:

- view-dashboard — platform stats overview
- view-events — full event catalogue with search & filters
- view-detail — single event detail and registration
- view-create — create or edit event form

3. Feedback & UX:

- Toast notifications for all success and error states
- Confirmation modal before any destructive action (delete event)

JavaScript Requirements:

- All views live in a single index.html; JS controls visibility with show/hide logic.
- Implement a central router in app.js that maps views to loader functions.
- No full page reloads — navigation is handled entirely in JavaScript.

CSS Requirements:

- Navigation bar fixed at the top with clear active-state styling.
- Toast notifications styled for both success (green) and error (red) states.
- Confirmation modal overlay styled and centered on the screen.
- Fully responsive across mobile and desktop.

Deliverables

Repositories:

- event-platform-backend repository with a working README.md and .env.example file.
- event-platform-frontend repository with a working README.md.
- node_modules/ excluded via .gitignore in both repositories.

Demo Video:

- Record a walkthrough demo of the full app with all views working.
- The video must show: Dashboard, Events List, Event Detail, Create Event, Edit Event, and Registration flow.

Thank You
Edges For Training Team